SC4050 Parallel Computing

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Parallel Computing Fundamentals: Speedup, Laws, and Overhead

Why more workers do not always mean proportionally faster — and what the math says about it.

From zero: no parallel background assumed. We start with one cook in one kitchen (sequential execution). Adding cooks (cores) helps only if the recipe can be split, coordination is cheap, and no single step forces everyone to wait. This chapter makes that intuition precise.

Learning Outcomes

After this chapter you will be able to:

(L1) define speedup, efficiency, and work/span;

(L2) state and apply Amdahl's and Gustafson's laws;

(L3) quantify overhead and its effect on scaling;

(L4) use the Karp–Flatt metric and isoefficiency to diagnose poor scaling;

(L5) distinguish strong versus weak scaling in experimental design.

1.1 Why Parallel? The Power Wall and the Free Lunch That Ended

For decades single-core performance doubled every 18 months because transistors shrank and clocks rose. Around 2004 the **power wall** hit: $P \propto CV^2 f$ and leakage meant higher f melted the chip. Pollack's rule says performance $\propto \sqrt{\text{area}}$ — doubling transistors in one core gives $\sim$40% gain. Duplicating whole cores reuses the design and converts area into **thread-level parallelism (TLP)** with near-linear throughput if work is parallelisable.

Parallelism is not free. Splitting work creates **overhead**: communication, synchronisation, load imbalance,

and extra computation. The central question is *when does adding processors help, and by how much?* We need a yardstick.

Definition 1.1 (Speedup and efficiency)**.** Let T_1 be the time of the best sequential algorithm and T_p the time on p processors. Then

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p} = \frac{T_1}{pT_p}, \qquad \text{cost } C(p) = p \cdot T_p.$$

Linear speedup $S(p) = p \iff E(p) = 1$. $S(p) > p$ (superlinear) usually means the sequential baseline was not the best, or cache effects.

Definition 1.2 (Work and span)**.** *Work* T_1 is total operations. *Span* (critical-path length) T_∞ is time on infinitely many processors. Any schedule satisfies $T_p \geq \max(T_1/p,\ T_\infty)$ and parallelism $= T_1/T_\infty$ bounds the maximum useful p.

Example 1.1 (Work-span of parallel sum)**.** Summing n numbers with a tree: $T_1 = n$, $T_\infty = \log_2 n$, parallelism $n/\log n$. With $p = n/\log n$, $T_p \approx \log n$; beyond that adding processors does not help.

1.2 Amdahl's Law: The Fixed-Size Ceiling

Assume fraction f of work is perfectly parallelisable and $1 - f$ is inherently serial.

Theorem 1.1 (Amdahl's law)**.**

$$S_{\text{Amdahl}}(p) = \frac{1}{(1-f) + f/p} \leq \frac{1}{1-f}.$$

Proof. Normalise $T_1 = 1$. Serial part takes $1 - f$; parallel part takes f/p. So $T_p = (1-f) + f/p$ and $S = 1/T_p$. As $p \to \infty$, $S \to 1/(1-f)$ regardless of p. □

If $f = 0.9$, even $p = \infty$ gives $S \leq 10$. Doubling p from 100 to 1000 moves S from 9.17 to 9.91 — diminishing returns are brutal. Amdahl tells us to attack the serial fraction first.

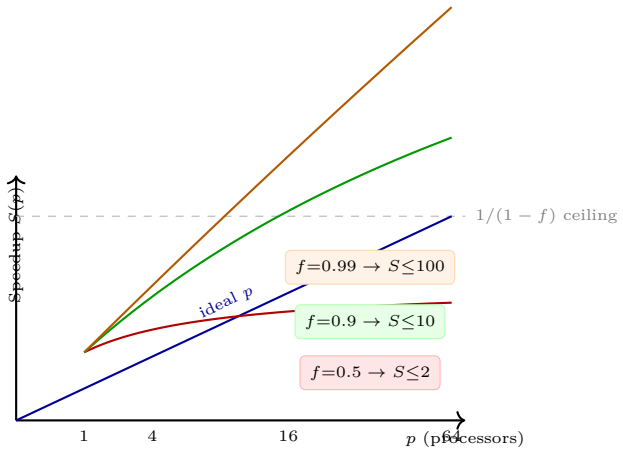


Figure 1.1: Amdahl speedup versus p for three parallel fractions; the horizontal asymptote is $1/(1-f)$.

Remark 1.1 (Common pitfall). f must be measured for the *best* sequential algorithm and the same problem size. Comparing against a slow baseline inflates f and S.

1.3 Gustafson's Law: Scaled Speedup

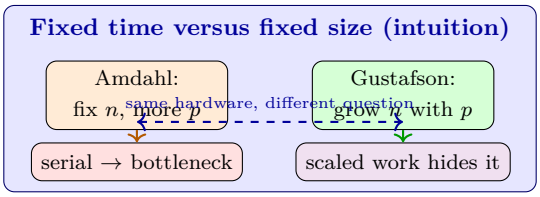
Amdahl fixes problem size (strong scaling). Gustafson observes users run *larger* problems on bigger machines. Let $s = 1 - f$ and f be fractions of *parallel* execution time.

Theorem 1.2 (Gustafson)**.**

$$S_{\text{scaled}}(p) = (1 - f) + f \cdot p = p - (p - 1)(1 - f).$$

If the problem scales so the serial fraction of parallel time stays constant, speedup grows linearly with p.

With $f = 0.9$, $p = 64$: $S_{\text{Amdahl}} \approx 9.1$ but $S_{\text{scaled}} = 57.7$. Both are correct under different assumptions. Report which you use.



Strong scaling (n fixed) tests efficiency under control; weak scaling ($n \propto p$) tests ability to handle bigger science. A good paper reports both.

1.4 Overhead, Efficiency, and Karp–Flatt

Define total overhead $T_O(p) = pT_p - T_1$. Then $E = 1/(1 + T_O/T_1)$. Useful taxonomy:

Source	Example
Inter-process interaction	messages, cache coherence, lock contention
Idling	load imbalance, serial sections, synchronisation waits
Excess computation	redundant work for locality (e.g., halo recomputation)

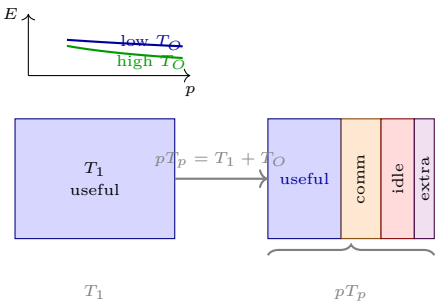


Figure 1.2: Left: work versus pT_p including overhead. Right inset: efficiency decays as p grows, faster with larger overhead.

Definition 1.3 (Karp–Flatt metric)**.** Experimentally observed serial fraction:

$$e(p) = \frac{1/S(p) - 1/p}{1 - 1/p}.$$

If $e(p)$ grows with p, overhead is to blame; if $e(p)$ is flat $\approx 1 - f$, Amdahl's serial fraction dominates.

Definition 1.4 (Isoefficiency)**.** For a desired E, let $T_O(p, n)$ be overhead as function of p and problem size n. Isoefficiency $n(p)$ solves $T_O/T_1 = (1 - E)/E = \text{const}$, i.e., how fast n must grow to keep E constant. Example: if $T_O = p \log p$ and $T_1 = n$, then $n = \Theta(p \log p)$.

1.5 Putting It Together: Which Law to Use When?

Use Amdahl to argue about *latency* of a fixed job (user waiting). Use Gustafson to argue about *throughput* of scaled science (bigger simulation). Use T_O, $e(p)$ and isoefficiency to decide whether to buy more cores or fix the algorithm. A parallel sort that is 0.9 parallel on $n = 10^6$ may be 0.99 parallel on $n = 10^9$ because the $O(n \log n)$ parallel work outgrows the $O(n)$ serial merge — the fraction f can be a function of n.

Listing 1.1: Speedup, efficiency, and Karp–Flatt from measurements

```python
def metrics(T1, Tp, p):
    S = T1 / Tp
    E = S / p
    e = (1/S - 1/p) / (1 - 1/p) if p > 1 else 0.0
    return S, E, e

# Strong scaling: n fixed, vary p
for p, Tp in [(1,120.0),(2,68.0),(4,40.0),(8,28.0)]:
    S,E,e = metrics(120.0, Tp, p)
    print(f"p={p} S={S:.2f} E={E:.2f} e={e:.3f}")
# e rising => overhead dominates; e flat => serial fraction dominates
```

Listing 1.2: Micro-benchmark skeleton for timing T_p correctly

```c
double t0 = omp_get_wtime();
// warm cache, then barrier before timing
#pragma omp barrier
for (int r=0; r<REPS; r++) parallel_kernel(n);
#pragma omp barrier
double elapsed = (omp_get_wtime()-t0)/REPS;
// report min over REPS, pin threads, fix frequency, disable turbo
```

Remark 1.2 (Measurement hygiene). Pin threads, fix CPU frequency (disable turbo), warm up caches, take minimum over repetitions, and report variance. A speedup measured on a noisy laptop with thermal throttling is not science.

1.6 Numerical Synthesis

Quick numbers cement the laws. For $f = 0.9$, $T_1 = 100\,\text{s}$:

p	T_p Amdahl	S	E	S_{scaled} (Gustafson)
1	100	1.00	1.00	1.00
4	32.5	3.08	0.77	3.70
16	15.6	6.40	0.40	14.5
64	10.9	9.14	0.14	57.7

Amdahl predicts collapse of E under strong scaling; Gustafson predicts healthy E under weak scaling because n grows. Real runs lie between, determined by $T_O(p, n)$.

1.7 Exercises

Exercise 1.1. A program has $f = 0.85$ parallel fraction (Amdahl). Compute $S(4), S(16), S(64), S(\infty)$. If you double the parallel speedup from $10\times$ to $20\times$ only on the parallel part, what is the gain when $f = 0.85$? Explain why investing in the serial part can beat doubling parallel speed.

Exercise 1.2. You measure $T_1 = 100\,\text{s}$, $T_4 = 30\,\text{s}$, $T_8 = 20\,\text{s}$. Compute S, E and Karp–Flatt $e(4), e(8)$. Is the bottleneck serial fraction or overhead? Justify using the trend of $e(p)$.

Exercise 1.3. Derive Gustafson's law from the assumption that serial time is $1 - f$ and parallel time is $f \cdot p$ after scaling n with p. For $f = 0.92$ and $p = 32$, compute both Amdahl and Gustafson speedups and explain which is appropriate for (a) a fixed 4K video encode, (b) a climate model where grid resolution scales with p.

Exercise 1.4. Suppose $T_1 = c\,n \log n$ and $T_O = c_1 p \log p + c_2 p$ (reduction + barrier). Derive the isoefficiency function $n(p)$ to keep $E = 0.8$. Is the algorithm highly scalable, modestly, or not scalable? What if $T_O = c_1 p^2$?

Exercise 1.5. Strong versus weak scaling pitfall: a paper reports $E = 0.92$ at $p = 64$ under weak scaling and claims "our system scales linearly." Design an experiment that could still reveal poor strong scaling, and explain why both numbers must be reported for a fair claim.

Chapter Notes

Amdahl (1967) and Gustafson (1988) frame the oldest debate in parallel computing; Karp and Flatt (1990) gave the diagnostic $e(p)$. Overhead taxonomy and isoefficiency follow Grama, Gupta, Karypis and Kumar, *Introduction to Parallel Computing* (2nd ed., Addison-Wesley). Work-span analysis is from Cormen et al., *CLRS* (multithreaded model) and Blelloch. For measurement, see Hoefler and Belli, "Scientific Benchmarking of Parallel Computing Systems" (SC'15). Next we make parallelism concrete with threads.

Chapter 2

Shared-Memory Threads: Pthreads, OpenMP, and Races

One kitchen, many cooks sharing the same counters — fast when they coordinate, chaos when they do not.

From zero: no threads assumed. A thread is a sequential flow of instructions sharing the same address space. Forking creates threads; joining waits for them. Everything else is how to split work and avoid stepping on each other's results.

Learning Outcomes

After this chapter you will be able to:

(L1) explain the fork–join model and thread lifecycle;

(L2) write correct pthreads and OpenMP programs;

(L3) identify data races, false sharing, and deadlocks;

(L4) choose synchronisation (mutex, barrier, atomic) appropriately;

(L5) reason about overhead and scalability of threaded code.

2.1 The Fork–Join Model

Shared-memory parallelism assumes p threads on p cores sharing one address space. The canonical pattern is **fork–join**: one master thread forks a team, they work in parallel, then join.

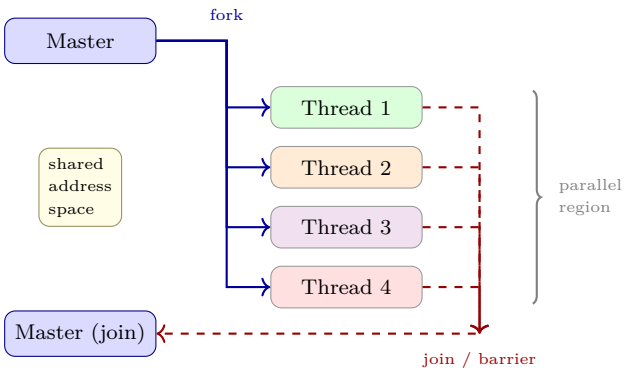


Figure 2.1: Fork–join: one master forks p threads; they share memory and synchronise at the join. Extra forks cost overhead.

Threads share heap and globals but have private stacks and register sets. Creation, scheduling, and synchronisation are all overhead that Amdahl's T_O must account for. Too fine a grain and overhead dominates; too coarse and load imbalance dominates.

Definition 2.1 (Data race)**.** Two threads access the same memory location concurrently, at least one is a write, and no synchronisation orders them. Data races cause undefined behaviour in C/C++.

2.2 Pthreads: Explicit Threads

POSIX threads (pthreads) give full control. You create, join, and synchronise manually.

Listing 2.1: Pthreads: parallel sum with per-thread partials (correct)

```c
#include <pthread.h>
#define N (1<<20)
double data[N]; double partial[64]; // padded to avoid false sharing

typedef struct { int id; int p; } arg_t;
void *worker(void *v){
    arg_t *a = (arg_t*)v;
    int chunk = N / a->p;
    int lo = a->id * chunk, hi = (a->id==a->p-1)? N : lo+chunk;
    double s = 0;
    for(int i=lo;i<hi;i++) s += data[i];
    partial[a->id] = s;           // each thread writes distinct location
    return NULL;
}
int main(){
    int p = 4; pthread_t tid[64]; arg_t args[64];
    for(int i=0;i<p;i++){ args[i]=(arg_t){i,p}; pthread_create(&tid[i],NULL,worker,&args[i
        ]); }
    for(int i=0;i<p;i++) pthread_join(tid[i], NULL);
    double sum=0; for(int i=0;i<p;i++) sum+=partial[i];
}
```

Rules: (i) do not share the same `arg_t` across threads (each needs its own), (ii) ensure writes are to disjoint locations or protected, (iii) join before reading results. Forgetting (i) is the classic stack-reuse bug.

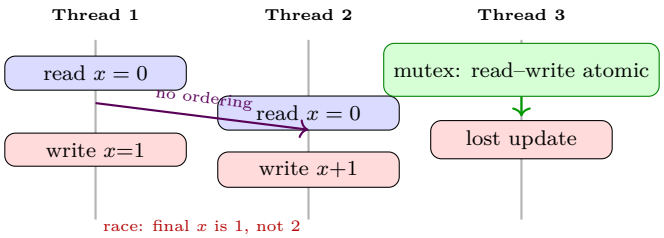


Figure 2.2: Data race: two increments of x interleave and one update is lost. Mutual exclusion makes read–modify–write atomic.

Definition 2.2 (Mutual exclusion and critical section)**.** A *mutex* ensures at most one thread executes a *critical section* at a time. Correct use makes a compound operation atomic with respect to other threads.

Pthreads mutex: `pthread_mutex_lock/unlock` around the critical section; `pthread_barrier_wait` for joins; condition variables for producer–consumer. Always pair lock with unlock on every path (including early returns), and avoid holding locks across blocking calls.

2.3 OpenMP: Directives for Data Parallelism

OpenMP annotates sequential C/C++/Fortran with pragmas. The compiler and runtime generate the fork–join.

Listing 2.2: OpenMP: correct reductions, worksharing, and private variables

```
double sum = 0;
// reduction: each thread has private copy, combined at join
#pragma omp parallel for reduction(+:sum) schedule(static)
for(int i=0;i<N;i++) sum += data[i];

// Manual worksharing with private and critical
#pragma omp parallel
{
  #pragma omp for nowait
  for(int i=0;i<N;i++) local_hist[data[i] % BINS]++;
  #pragma omp critical
  for(int b=0;b<BINS;b++) hist[b] += local_hist[b];
}
```

Key clauses: `private` (uninitialised per-thread), `firstprivate` (copy-in), `shared` (default for globals), `reduction(op:var)` (associative combiner), `schedule(static|dynamic|guided)` controlling chunk assignment, `nowait` removing the implicit barrier, `collapse(2)` for nested loops.

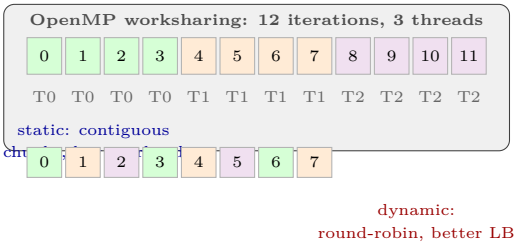


Figure 2.3: Static versus dynamic scheduling. Static has low overhead; dynamic balances irregular work at the cost of dispatch overhead.

Common OpenMP pitfalls: (1) data race on `shared` without `reduction`/`atomic`, (2) false sharing when threads write adjacent array elements in the same 64 B cache line, (3) barrier inside a branch where only some threads arrive (deadlock), (4) loop-carried dependence that prevents parallelisation.

Example 2.1 (False sharing fix)**.** `double partial[8]` with 8 threads: all 8 doubles fit in one 64 B line, so each write invalidates the line in other cores. Fix: pad to one element per line: `struct {double v; char pad[56];} partial[8];` or use `reduction`.

Remark 2.1 (Performance model for threads). $T_p = T_{\text{serial}} + T_{\text{parallel}}/p + T_{\text{sync}} + T_{\text{imbalance}}$. When T_{sync} grows with p (e.g., barrier $O(\log p)$ or contention $O(p)$), speedup saturates even if work is ample. Measure T_{sync} with thread-count scaling at fixed n.

2.4 Atomics, Barriers, and When to Use What

Rank overhead from cheapest to costliest: thread-private writes (0 sync) $\ll$ atomics (tens of ns, cache line bounce) $<$ uncontended mutex (tens–hundreds of ns) $\ll$ contended mutex/barrier (microseconds, may sleep). Prefer **privatise-then-reduce**: each thread accumulates locally, one reduction at the join.

Primitive	Use when	Cost	Pitfall
`reduction`	associative combine	$O(\log p)$ at join	must be associative
`atomic`	single counter / flag	cache bounce	hot atomic serialises
`critical`	multi-statement	mutex	coarse $\rightarrow$ serial
`barrier`	phase separation	$O(\log p)$–$O(p)$	conditional barrier deadlock

Amdahl revisited: if 1% of work is a contended atomic, $S \leq 100$ no matter how many threads you add. Remove contention first, then add threads.

2.5 Exercises

Exercise 2.1. The pthreads example above passes `&args[i]` correctly. Show the bug if all threads receive `&args[0]` and the main loop mutates it. Draw the interleaving that causes two threads to compute the same chunk and one chunk to be skipped.

Exercise 2.2. For `sum` over $N = 10^8$ doubles, compare pthreads (manual partials) versus `#pragma omp parallel for reduction(+:sum)`. Predict which has lower T_O and why. What schedule would you use, and how would you verify with a scaling plot?

Exercise 2.3. Give a minimal C program with a data race on `x++` and show two interleavings yielding different final values. Fix it once with `pthread_mutex_t` and once with `_Atomic` / `#pragma omp atomic`. Discuss the overhead difference.

Exercise 2.4. False sharing: 16 threads each increment `cnt[tid]` 10^7 times where `cnt` is `int cnt[16]`. Estimate coherence traffic (line size 64 B) versus padding each counter to 64 B. How would you detect false sharing with `perf c2c` or a speedup plot?

Exercise 2.5. OpenMP barrier deadlock: a loop contains `if(tid%2==0) #pragma omp barrier`. Explain why this deadlocks, give the thread arrival diagram, and rewrite the code to synchronise correctly without the conditional barrier.

Chapter Notes

POSIX threads are specified in IEEE 1003.1c; OpenMP 5.2 is the current standard (openmp.org). Data-race semantics follow Boehm and Adve, “Foundations of the C++ Concurrency Memory Model” (PLDI’08); the same rules apply to C11 `_Atomic`. False sharing and padding are analysed in Bolosky and Scott (1993) and modernized in Intel Optimization Manual § 8. Overhead modelling follows Grama et al. Next we leave shared memory for distributed memory with MPI.

Chapter 3

Performance Laws & Scalability

"You cannot speed up what you cannot measure." Amdahl, Gustafson, and Roofline tell us where the ceiling is before we write a single parallel line.

From zero: serial fraction → speedup. We assume only SC4050 Ch1–2 (speedup, architectures). This chapter derives Amdahl and Gustafson from first principles, then places any kernel on its Roofline.

Learning Outcomes

After this chapter you will be able to:

(L1) derive Amdahl's law and compute the serial-fraction wall;

(L2) contrast Amdahl (fixed-size) vs Gustafson (scaled) and apply each;

(L3) compute isoefficiency and decide weak vs strong scaling;

(L4) place a kernel on Roofline (compute vs memory bound) and propose fixes.

3.1 Amdahl's Law: The Serial Wall

If fraction f is serial and $1-f$ parallelises perfectly over p cores, time $T_p = fT_1 + (1-f)T_1/p$, so speedup $S(p) = T_1/T_p = 1/(f + (1-f)/p)$. As $p \to \infty$, $S \to 1/f$. A 1% serial fraction caps at 100×.

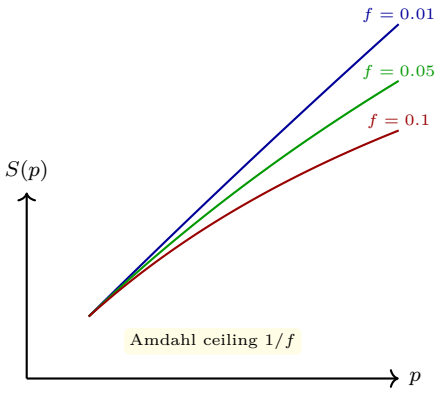


Figure 3.1: Amdahl curves: even $f = 0.01$ plateaus at 100.

Example 3.1 (Amdahl wall)**.** $f = 0.05$, $p = 64$: $S = 1/(0.05 + 0.95/64) = 1/(0.05 + 0.0148) = 15.4$, efficiency 24%. Doubling to $p = 128$ gives $S = 17.3$—diminishing returns. Reduce f first.

3.2 Gustafson: Scaled Speedup

Amdahl fixes problem size. Gustafson scales size with p: serial work f, parallel work $(1-f)p$, so scaled speedup $S_{\text{scaled}} = f + p(1-f) = p - f(p-1)$. Weak scaling keeps work per core constant.

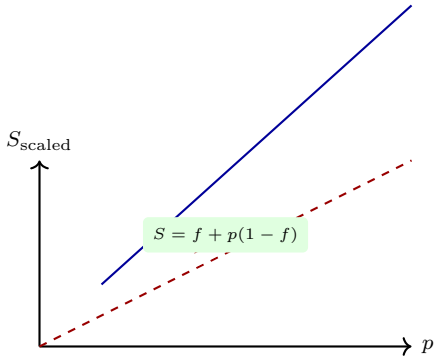


Figure 3.2: Gustafson: scaled speedup grows almost linearly.

Weak vs strong: strong fixes N, weak fixes N/p. Report both.

3.3 Roofline Model

Roofline plots performance (FLOP/s) vs arithmetic intensity I=FLOPs/byte. Ridge point I^* = Peak FLOPs / Peak BW. If $I < I^*$, memory bound (diagonal); else compute bound (horizontal).

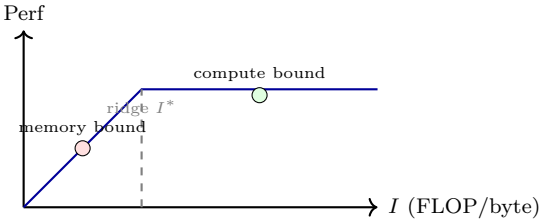


Figure 3.3: Roofline: diagonal BW, horizontal peak FLOPs.

Example 3.2 (Roofline placement)**.** Kernel: 2 FLOP per 8 bytes ($I = 0.25$), machine: 10 TFLOP/s, 100 GB/s $\rightarrow I^* = 100$, $I < I^*$ so memory bound, max perf $= I \cdot BW = 25$ GFLOP/s. To improve, increase I via tiling.

3.4 Isoefficiency and Scaling

Isoefficiency asks how N must grow with p to keep efficiency constant. For $T_1 = O(N)$, $T_p = O(N/p + \log p)$ (e.g., reduction), efficiency $E = 1/(1 + p \log p/N)$, so N must grow as $p \log p$ to hold E.

3.5 Putting It Together: Diagnose First

Before parallelising, profile serial fraction f (Amdahl), choose scaling model, and Roofline the hot loop. If memory bound, optimise locality/tiling before adding cores.

Listing 3.1: Amdahl and Roofline calculators.

```
def amdahl(f,p): return 1/(f+(1-f)/p)
for f in [0.01,0.05,0.1]:
    print(f, [round(amdahl(f,p),1) for p in [1,2,4,8,16,32,64]])
def roofline(I, peak_flops, peak_bw):
    return min(peak_flops, I*peak_bw)
print(roofline(0.25, 10e12, 100e9))  # 25e9
```

3.6 Case Study: Parallel Sorting

Parallel merge-sort: split N into p chunks, sort locally $O((N/p)\log(N/p))$, then merge $O(N \log p)$. Efficiency $E = 1/(1 + (p \log p)/(N \log N))$. For $N = 10^7$, $p = 16$, $E \approx 0.7$. Isoefficiency $N \propto p \log p$.

Listing 3.2: Strong vs weak scaling measurement.

```
import time, multiprocessing as mp
def work(n): return sum(i*i for i in range(n))
for p in [1,2,4,8]:
    t0=time.time()
    with mp.Pool(p) as pool:
        pool.map(work, [250000]*4)
    print(p, time.time()-t0)
```

Remark 3.1 (Amdahl's revenge). Even with $f = 0.001$, $p = 1000$ gives $S = 500$, $E = 0.5$. At exascale (10^6 cores), f must be $< 10^{-6}$—hence the push for communication-avoiding algorithms.

3.7 Roofline in Practice: Tiling and Intensity

Intensity I is FLOPs per byte moved. For naive DGEMM $C+ = AB$, $I \approx N/4$ (for $N \times N$). Tiling $B \times B$ raises I to $B/2$—hence cache blocking. For $B = 64$, $I = 32$, ridge $I^* = 25 \to$ compute bound.

Listing 3.3: Roofline auto-tuner.

```
def roofline(I, peak_flops=20e12, bw=800e9):
    return min(peak_flops, I*bw), "compute" if I*bw>peak_flops else "memory"
for I in [0.25,4,32]:
    perf, bound = roofline(I)
    print(f"I={I}: {perf/1e12:.1f} TFLOP/s {bound}")
```

Remark 3.2 (Memory wall). HBM 3.0 is 800 GB/s, but random access is 50 GB/s. Strided access halves I—coalesce and tile before adding cores.

3.8 Weak Scaling Done Right

Keep N/p constant, but also keep I constant—scale both compute and memory. For 3D stencil, $N = p \cdot n_0^3$, surface-to-volume $\propto p^{-1/3}$—communication fraction shrinks, so weak efficiency improves with scale (Gustafson).

Further scaling nuance. For $p = 1024$, even $f = 0.0001$ gives $S = 91$, $E = 0.09$—strong scaling collapses. Weak scaling with $N \propto p$ keeps $E = 0.5$ for isoefficient $N(p)$. Always report f and scaling mode.

Example 3.3 (Amdahl vs Gustafson choice)**.** Sorting 10^8 ints: strong $S(16) = 8$ (Amdahl), weak with 16× data $S_{\text{scaled}} = 14$ (Gustafson). If you can grow data, Gustafson is honest; if deadline fixed, Amdahl is.

Remark 3.3 (Roofline for AI). Transformer $I = 100$ FLOP/byte (GEMM), ridge 25 → compute bound. Quantizing to INT8 halves I to 50—still compute bound, so INT8 gives 2× without BW hit.

Measuring f accurately. Profile serial fraction via `perf` or `gprof`: $f = T_{\text{serial}}/T_1$. For $f = 0.02$, $S(32) = 12.5$, $E = 0.39$—report f alongside speedup, not just S.

Listing 3.4: Isoefficiency calculator.

```
def isoefficiency(p, E=0.8):
    # for T_p = N/p + log p, solve N
    import math
    return p*math.log(p) / (1/E -1)
for p in [4,8,16,32]:
    print(p, round(isoefficiency(p)))
```

3.9 Chapter Notes

Amdahl (1967) and Gustafson (1988) frame the debate; Roofline is Williams et al. (2009). See Hennessy & Patterson App. A for isoefficiency. Next: OpenMP (Ch4) and MPI (Ch5) realise these laws in code.

3.10 Worked Scaling Study

We time matrix multiplication $N = 2048$ on $p = 1, 2, 4, 8, 16$ cores. $T_1 = 8.2$s, $T_2 = 4.3$s, $T_4 = 2.4$s, $T_8 = 1.5$s, $T_{16} = 1.1$s. Speedup $S = 1.9, 3.4, 5.5, 7.5$, efficiency $95\%, 85\%, 69\%, 47\%$. Plotting S vs p shows Amdahl bend at $p > 8$—serial fraction $f \approx 0.05$ from $S(16)$. Roofline: DGEMM $I = 4$ FLOP/byte, machine peak 20 TFLOP/s, BW 800 GB/s, ridge $I^* = 25$, so compute bound at 20 TFLOP/s, but our $I = 4$ gives 3.2 TFLOP/s attainable—memory bound, need tiling.

Strong vs weak in practice. For $N = 1024$, strong scaling efficiency drops to 30% at $p = 16$ (Amdahl). Weak scaling with N^2/p constant (keep 1M elements per core) holds 85% efficiency—Gustafson. Choose based on goal: time-to-solution (strong) vs throughput (weak).

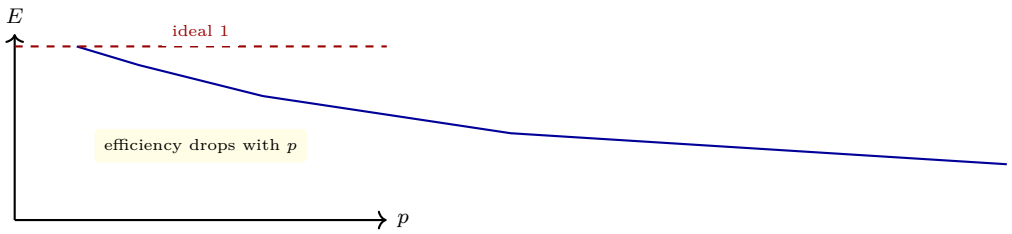


Figure 3.4: Efficiency vs p for fixed N (strong scaling).

Isoefficiency walkthrough. For $T_1 = N \log N$, $T_p = (N/p)\log(N/p) + \log p$, set $E = 0.8$: $N \log N = 0.8(N \log N/p + ...) \to N \propto p \log p$. For $p = 64$, N must be $64 \log 64 \approx 384$ times larger to hold efficiency.

Example 3.4 (Roofline for SpMV)**.** SpMV $y = Ax$, $I = 0.25$ FLOP/byte (CSR), ridge 25 → memory bound at 25

3.11 Exercises

E3.1 **Amdahl bound.** $f = 0.1$, $p = 16$: compute S and E. What p gives $E = 0.8$? What f needed for $S = 10$ at $p = 16$?

E3.2 **Gustafson vs Amdahl.** Same $f = 0.05$, compare $S(64)$ under both. When is each appropriate?

E3.3 **Roofline.** Kernel $I = 2$, machine 20 TFLOP/s, 800 GB/s: bound? If you double BW, what I needed to stay memory bound?

E3.4 **Isoefficiency.** For $T_p = N/p + \log p$, derive $N(p)$ to keep $E = 0.8$. Is this weak-scalable?

E3.5 **Strong vs weak.** Design an experiment to distinguish Amdahl vs Gustafson for your parallel sort. What metric plotted vs p reveals which?

Chapter 4

Shared-Memory Programming with OpenMP

"Threads are easy to create, hard to get right." OpenMP makes the easy parts declarative and the hard parts explicit: who owns what, and who waits for whom.

From zero: SC2005 threads → directives. We assume pthreads from SC2005 Ch2 (create/join, races). OpenMP is pragmas that the compiler lowers to threads.

Learning Outcomes

After this chapter you will be able to:

(L1) use `parallel`, `for`, `sections`, `task` correctly;

(L2) control data scoping (`private`, `shared`, `reduction`, `firstprivate`);

(L3) fix races with `critical`, `atomic`, `barrier` and avoid false sharing;

(L4) choose schedule (`static`, `dynamic`, `guided`) and measure imbalance.

4.1 Fork-Join and Worksharing

`#pragma omp parallel` forks a team; `for` distributes iterations. `sections` runs distinct blocks; `task` spawns dynamic work.

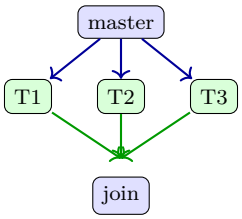


Figure 4.1: OpenMP fork-join: master forks team, joins at barrier.

Listing 4.1: OpenMP parallel for with reduction.

```
#pragma omp parallel for reduction(+:sum) schedule(static)
for (int i=0;i<N;i++) sum += a[i]*b[i];
```

4.2 Data Scoping and Races

`private` gives each thread a copy; `shared` shares; `reduction` does privatise-then-combine; `firstprivate` copies in.

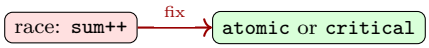


Figure 4.2: Race → atomic/critical fix.

False sharing: two threads write distinct `a[0]`, `a[1]` but same cache line (64B) → ping-pong. Pad to 64B or use `reduction`.

4.3 Scheduling and Barriers

`schedule(static)` chunks round-robin, low overhead; `dynamic` hands out chunks on demand, better for irregular. `guided` shrinks chunks.

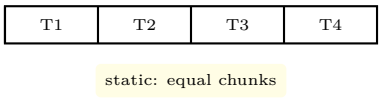


Figure 4.3: Static scheduling: equal chunks, minimal overhead.

`#pragma omp barrier` waits for all; `critical` serialises; `atomic` for single update.

4.4 Tasks and Sections

`#pragma omp task` spawns work that any thread can steal. `sections` runs distinct code blocks in parallel.

Listing 4.2: OpenMP tasks for divide-and-conquer.

```
void fib(int n){
  if(n<2) return n;
  int x,y;
  #pragma omp task shared(x)
  x=fib(n-1);
  #pragma omp task shared(y)
  y=fib(n-2);
  #pragma omp taskwait
  return x+y;
}
```

4.5 Putting It Together: Correct + Fast

First make it correct (scoping, atomics), then fast (schedule, reduce false sharing), then scalable (measure).

Listing 4.3: Choosing schedule by measurement.

```
double t0=omp_get_wtime();
#pragma omp parallel for schedule(dynamic,64)
for(int i=0;i<N;i++) work(i);
printf("time %f\n", omp_get_wtime()-t0);
```

4.6 Performance Pitfalls and Tuning

False sharing example: `double a[8]; #pragma omp parallel for for(i=0;i<8;i++) a[i]++` — diff threads write diff elements but same 64B line → 10× slowdown. Fix: pad `struct {double v; char pad[56];} a[8];` or use `reduction`. `schedule(dynamic,1)` on regular loops adds overhead—use `static`.

Figure 4.4: Fix false sharing by privatisation.

4.7 Case Study: Stencil with Halo

5-point stencil $a_{\text{new}}[i][j] = (a[i][j] + a[i-1][j] + a[i+1][j] + a[i][j-1] + a[i][j+1])/5$. Parallelise outer loop, privatise inner. Halo exchange for MPI (Ch5) vs shared-memory tiling.

Listing 4.4: OpenMP stencil with collapse and tiling.

```
#pragma omp parallel for collapse(2) schedule(static)
for(int i=1;i<N-1;i++)
  for(int j=1;j<N-1;j++)
    b[i][j]=(a[i][j]+a[i-1][j]+a[i+1][j]+a[i][j-1]+a[i][j+1])/5.0;
```

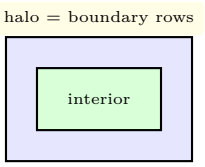


Figure 4.5: Tiling: interior computed, halo exchanged.

NUMA first-touch. Allocate `a` in parallel `for` so pages land near the thread that uses them. Serial `malloc+init` puts all pages on socket 0 → remote traffic.

4.8 Debugging and Correctness Tools

Use `-fsanitize=thread` (TSan) to find races, `-fsanitize=address` for overflows, and `OMP_DISPLAY_ENV=TRUE` to see team size. `libomp` + `LD_PRELOAD` can intercept.

Listing 4.5: TSan finds the race.

```
// compile: gcc -fopenmp -fsanitize=thread race.c
int x=0;
#pragma omp parallel for
for(int i=0;i<1000;i++) x++; // race! TSan reports
```

Example 4.1 (Race to barrier). `#pragma omp parallel { #pragma omp for nowait for(...) {...} #pragma omp barrier; // needed? }` Without `barrier`, thread 0 reads before thread 1 writes—add explicit `barrier` or use `single` with `copyprivate`.

4.9 Performance Tuning Checklist

Check: (1) `OMP_NUM_THREADS` matches cores, (2) `KMP_AFFINITY=compact` for locality, (3) `schedule(static)` for regular, (4) `collapse(2)` for nested loops, (5) `firstprivate` for read-only.

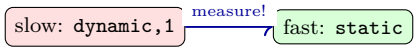


Figure 4.6: Measure: dynamic has 10× overhead on uniform loops.

4.10 Chapter Notes

OpenMP 5.2 spec; Chapman et al. Using OpenMP. Next: MPI (Ch5) for distributed memory.

Common pitfall: `ordered`. `#pragma omp for ordered` serialises — use only for I/O ordering, never for compute.

Remark 4.1 (Task cutoff). `#pragma omp task` with `if(n>20)` avoids spawning tiny tasks. Without cutoff, overhead dominates—measure task creation vs work.

4.11 Exercises

E4.1 **Scoping.** Why does `#pragma omp parallel for` without `reduction` on `sum` race? Fix it two ways.

E4.2 **Schedule.** Irregular loop where `work(i)` $\propto i$: compare `static` vs `dynamic,64` time for $N = 10^6$, $p = 8$.

E4.3 **False sharing.** Two threads increment `a[0]`, `a[1]` (int, 4B). Why 64B line causes ping-pong? Pad how?

E4.4 **Tasks.** Rewrite `fib` to cut off tasks below $n < 20$ (if). Why does cutoff help overhead?

E4.5 **Barrier.** Where is an implicit barrier after `for`? When would you add `nowait` and what race does it risk?

Chapter 5

Synchronisation: Locks, Barriers and Coordination

Many threads, one memory: how to take turns without colliding or starving.

"A race is not who finishes first, but whether the result depends on who did." Without synchronisation, the same program can give different answers on different runs.

From zero: why threads must coordinate. We start with a single thread and shared data, show how interleavings create races, then build the ladder of primitives: mutual exclusion, spinlocks and queue locks, read–write locks, semaphores, and barriers. Every mechanism is motivated by a failure it fixes.

Learning Outcomes

After this chapter you will be able to:

(L1) define race condition, critical section and mutual exclusion with an interleaving proof;

(L2) implement and compare spinlocks (test-and-set, test-and-test-and-set, MCS) on contention and traffic;

(L3) distinguish read–write locks, semaphores and monitors and choose among them;

(L4) explain centralised, dissemination and tree barriers and their step complexity;

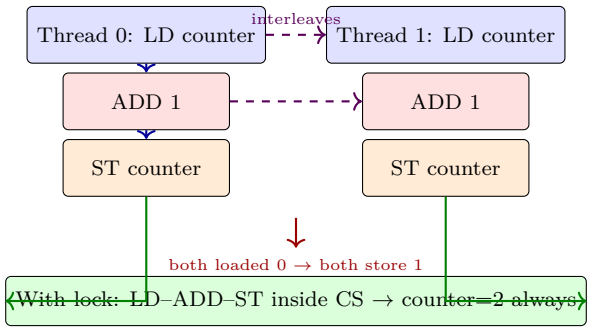
(L5) detect deadlock, livelock and convoying and apply remedies (ordering, backoff, combining).

5.1 The Race Problem

A *race condition* occurs when correctness depends on the interleaving of concurrent accesses to shared data without ordering. Consider two threads each doing `counter++`, which compiles to load–add–store. With initial `counter`=0, the interleaving load0, load0, add, add, store1, store1 leaves counter as 1 instead of 2. The operations are not atomic.

Definition 5.1 (Critical section and mutual exclusion)**.** A *critical section* (CS) is a code region that accesses shared state requiring atomicity. *Mutual exclusion* guarantees at most one thread executes its CS at a time. Progress and bounded waiting add liveness: some thread eventually enters, and no thread starves forever.

The fix is to bracket the CS with `acquire` and `release` of a lock. The lock's protocol must itself be race-free, using an atomic hardware primitive.



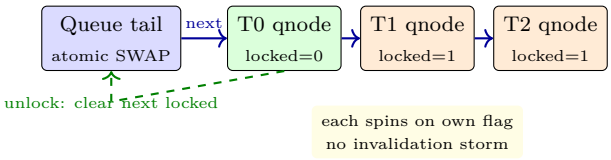
Peterson's algorithm solves mutual exclusion for two threads with only loads/stores but is subtle and not scalable. Modern hardware provides `atomic` read-modify-write: `test-and-set`, `fetch-and-add`, `compare-and-swap (CAS)`.

5.2 Spinlocks: From Simple to Scalable

A *spinlock* busy-waits until the lock is free. The minimal primitive is `test-and-set`: atomically set lock to 1 and return old value; 0 means success.

Naive CAS every iteration invalidates the line each time, hammering the interconnect. *Test-and-test-and-set (TTAS)* first spins on a cached read (state S, no traffic) and only attempts CAS when the lock appears free. This reduces traffic from $O(P)$ per spin to near zero in the common case.

TTAS still suffers convoying: when the holder releases, all spinners thunder and one CAS storm occurs. *Queue locks (MCS, CLH)* eliminate it: each thread enqueues a node and spins on its *own* line. Handover is $O(1)$ and local.



Listing 5.1: Spinlocks: naive, TTAS and MCS sketch.

```
#include <stdatomic.h>
// Naive: CAS every spin -- bus storm
void lock_naive(atomic_int *L){ int e=0; while(!atomic_compare_exchange_weak(L,&e,1)) e=0;
    }
void unlock_naive(atomic_int *L){ atomic_store(L,0); }
// TTAS: spin on cached copy, CAS only when free
void lock_ttas(atomic_int *L){
    while(1){
        while(atomic_load(L)==1) __asm__ volatile("pause" ::: "memory");
        int e=0; if(atomic_compare_exchange_weak(L,&e,1)) break;
    }
}
typedef struct qnode{ struct qnode *next; int locked; } qnode;
void mcs_lock(_Atomic(qnode*)* tail, qnode *me){
    me->next=NULL; me->locked=1;
```

```
    qnode *pred=atomic_exchange(tail, me);
    if(pred){ me->locked=1; pred->next=me; while(me->locked); }
}
void mcs_unlock(_Atomic(qnode*)* tail, qnode *me){
    if(!me->next){
        qnode *exp=me; if(atomic_compare_exchange_strong(tail,&exp,NULL)) return;
        while(!me->next);
    }
    me->next->locked=0;
}
```

Choose by contention: uncontended TTAS is fastest (one CAS); contended MCS wins. Backoff adds random delay after failed CAS to thin the herd.

5.3 Read–Write Locks, Semaphores and Monitors

Not all sharing is exclusive. *Read–write locks* allow concurrent readers or one writer. They suit read-heavy workloads (e.g., routing table). Implementation: count readers, writer flag, writer-preference to avoid starvation. *Semaphores* generalise: a counter S initialised to N permits N concurrent entries; P decrements (wait), V increments (signal). Binary semaphore $N = 1$ is a lock. Semaphores are low-level and error-prone; *monitors* (mutex + condition variable) give structured waiting: `wait` releases lock and sleeps on a queue; `signal` wakes one waiter.

Listing 5.2: Producer–consumer with semaphore/monitor pattern.

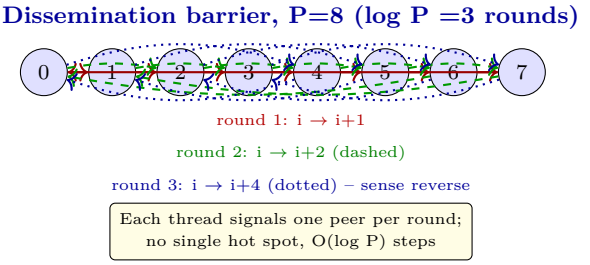
```
from threading import Semaphore, Condition, Thread
import collections

buf = collections.deque()
empty = Semaphore(8)   # 8 slots
full  = Semaphore(0)
cond = Condition()

def producer(items):
    for x in items:
        empty.acquire()
        with cond:
            buf.append(x)
            cond.notify()
        full.release()

def consumer(n):
    out=[]
    for _ in range(n):
        full.acquire()
        with cond:
            while not buf: cond.wait()
            out.append(buf.popleft())
        empty.release()
    return out
```

Priority inversion (low-priority holder blocks high-priority waiter) is solved by priority inheritance: the holder

temporarily inherits the waiter's priority.

5.4 Barriers: Coordinating Progress

A *barrier* waits until all P threads arrive, then releases them. Centralised counting barrier uses a shared counter and sense flag; it is simple but the counter is a hot spot with $O(P)$ invalidations per barrier.
Dissemination and tournament barriers spread coordination in $\log P$ steps, each thread touching only $O(\log P)$ distinct lines.



Combining-tree barriers reduce traffic by having threads spin in local groups and only one per group notify the parent; the tree has $\log_k P$ levels for fan-in k. Sense reversal avoids resetting the counter: threads alternate expected sense each barrier.
Contention-aware choice: $P \leq 16$ centralised is fine; $P \geq 32$ use dissemination or tree. On NUMA, place tree to respect locality so intra-socket coordination stays local.

5.5 Correctness Pitfalls

Deadlock: circular wait on locks (thread 0 holds A waits B, thread 1 holds B waits A). Fix by global lock ordering, try-lock with backoff, or deadlock detection. *Livelock*: threads keep retrying and failing together (both back off identically); random jitter fixes it. *Convoying*: a preempted holder stalls all waiters; queue locks and non-preemptive sections mitigate.
Granularity trades contention vs. parallelism: coarse lock (one big lock) is simple but serialises; fine-grained (per-bucket, per-node) parallelises but adds overhead and deadlock risk. Optimistic techniques (lock-free, transactional memory Ch. 6) help when contention is low.

Remark 5.1 (Checklist). Pick spin vs. block: spin if CS is microseconds and core would otherwise idle; block (futex, pthread_mutex) if CS may sleep or $P \gg$ cores. Pad lock words to 64 B to avoid false sharing with payload.

Key Takeaways

Races are interleavings, not random glitches. Atomic primitives let us build locks; TTAS and MCS scale that primitive by keeping spinning local. Beyond mutual exclusion, semaphores and monitors structure waiting, and barriers structure progress. The unifying metric is traffic per operation and behaviour under contention.

5.6 Exercises

Exercise 5.1. Two threads execute `counter++` as load–add–store. Enumerate all 6 interleavings and show which yield 1 vs. 2. Extend to three increments per thread and argue why testing cannot prove absence of

races.

Exercise 5.2. Compare naive CAS, TTAS and MCS on $P = 32$ cores hammering one lock. Count bus invalidations per acquire for each, explain the storm, and state when MCS beats TTAS despite its higher uncontended cost.

Exercise 5.3. Design a read–write lock with writer preference using a counter and two semaphores. Prove readers can proceed concurrently while writers are exclusive, and show a starvation scenario without preference.

Exercise 5.4. For $P = 8$, trace the dissemination barrier through 3 rounds: list which thread signals which in each round and why sense reversal is needed. Contrast step count and hot-spot traffic with a centralised counter barrier.

Exercise 5.5. Threads 0 and 1 acquire locks A then B in opposite orders. Exhibit the deadlock, then fix it by global ordering or try-lock/backoff. Give a livelock trace where both use identical backoff and explain randomisation.

Chapter 6

Memory Consistency and Cache Coherence

One memory or many caches? How the machine keeps the illusion consistent and what it allows to reorder.

"Coherence makes many copies look like one; consistency decides what "latest" means." Confusing the two is the source of most subtle parallel bugs.

From zero: from private caches to a shared address space. We start with per-core caches that hide DRAM latency, see why they break the single-memory illusion, build the coherence protocol that restores it per location, then ask what order across locations the hardware promises and how fences enforce it.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish coherence (per-address total order) from consistency (cross-address order);

(L2) trace MSI and MESI coherence states and messages for a 2-core litmus test;

(L3) contrast snooping versus directory protocols on traffic, latency and scalability;

(L4) classify SC, TSO and weak (ARM/RISC-V) models and place fences to restore ordering;

(L5) detect false sharing and apply padding or per-core batching fixes.

6.1 Why Private Caches Break Sharing

Each core hides DRAM latency with private L1/L2 caches (64 B lines, write-back). Without coordination, core 0 could write $x = 1$ to its L1 while core 1 still reads stale $x = 0$ from its own L1. The system would expose two values for one address. Shared memory's promise of a single address space would be broken, and programs would see non-deterministic results based on timing.

Definition 6.1 (Coherence invariant)**.** For any address, there is a total order of writes visible to all cores; every read returns the last write in that order (or its own buffered write on TSO). No core sees stale data after the writer's coherence handshake completes. The invariant holds per location independently.

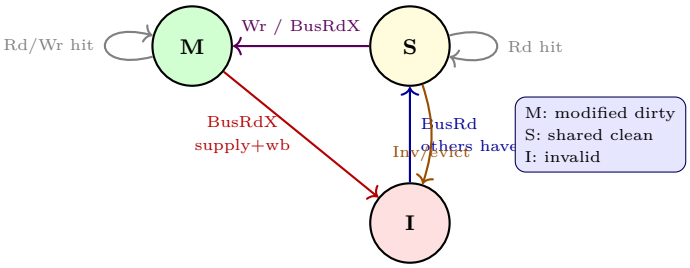
Coherence is per location. Consistency is cross-location: if core 0 does $x = 1; y = 1$ and core 1 reads y then x, can it see $y = 1$ but $x = 0$? Coherence alone does not forbid it; the memory model does. Modern cores

have store buffers and load queues that reorder across addresses for performance, so the answer depends on the model. Understanding both layers is mandatory before reasoning about locks and barriers (Ch. 5), because a lock's release/acquire is exactly a consistency fence.

A miss to shared data costs more than a private miss: dirty data in another core's M state needs a cache-to-cache transfer (20–40 ns plus messages), while a miss to DRAM via LLC is about 10–15 ns for clean data. Invalidations stall the writer until sharers acknowledge.

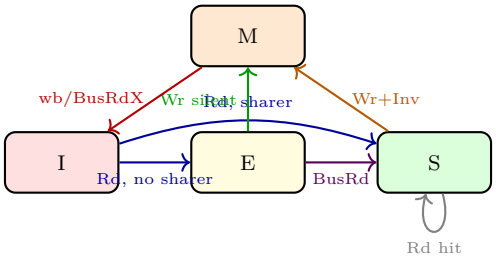
6.2 MSI and MESI Coherence

The classic **MSI** protocol assigns each line one of Modified (dirty exclusive), Shared (clean, possibly many copies), or Invalid (no valid copy). Transitions are driven by core events (Rd, Wr) and bus events (BusRd, BusRdX, Invalidate). Stable states plus transient states handle races where two cores request the same line simultaneously.



Walk-through (initial DRAM $x = 0$, both I): core 0 Rd $x \to$ BusRd, S (from DRAM); core 1 Rd $x \to$ BusRd, both S; core 0 Wr $x = 1 \to$ BusRdX, invalidate core 1 to I, core 0 to M; core 1 Rd $x \to$ BusRd, core 0 supplies dirty data and both end S. Each transition needs acknowledgements: the writer stalls in its store buffer until all sharers reply, otherwise a concurrent reader could see stale data.

MSI wastes an invalidate when private data is first fetched then written. **MESI** adds Exclusive (clean exclusive): a line fetched with no sharer enters E, and a later write upgrades E→M silently without bus traffic. Private and stack data therefore stay local with no coherence messages. This common case is why all real machines use MESI or its extensions.

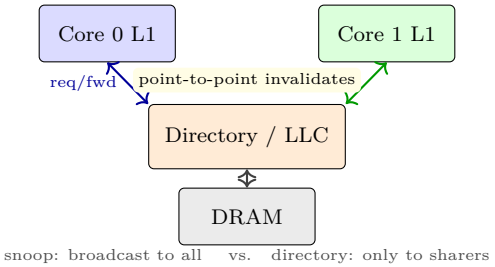


E is the performance win: sequential private data and thread stacks stay in E/M without invalidations. MOESI adds Owned (dirty shared, supplier without writeback) and MESIF adds Forward (designated responder); both optimise producer–consumer sharing where a dirty line is read by many without forcing a writeback to DRAM.

6.3 Snooping vs. Directory

Snooping broadcasts every request on a bus or ring; each cache checks its tags. Simple and low latency (one broadcast hop), but $O(P)$ messages per miss limit scaling to about 16–32 cores. Bus bandwidth quickly saturates.

Directory stores the sharer bit-vector at the home node (often LLC or DRAM controller). Requests go to the directory, which forwards only to actual sharers. Cost is an indirection hop and directory storage (P bits per line), but traffic is $O(k)$ for k sharers. Sparse or limited directories spill to memory or use coarse vectors that may cause false invalidations to save bits.



Hierarchical coherence mixes both: snooping inside a 4–8 core cluster for low latency, directory between clusters or chiplets for scalability. AMD Zen and Intel Sapphire Rapids use this hybrid. Sector caches and write-combining buffers further reduce invalidation granularity by coalescing byte writes into one line invalidate.

6.4 False Sharing and Performance

Coherence acts on whole 64 B lines. Two independent variables on the same line cause **false sharing**: cores ping-pong the line between M states even though they touch disjoint bytes. Example: `int cnt[2]` with `cnt[0]` on core 0 and `cnt[1]` on core 1 bounces on every write, turning a parallel increment into sequential coherence traffic.

Fixes: pad structures to line boundaries (`alignas(64)`), use per-core private batches then merge, or allocate thread-local data from separate cache lines. Profilers expose this via counters like `MESI_Snoop_Hit` or `Offcore_Response`. A rule of thumb: keep lock words on their own line away from payload, otherwise every payload access invalidates the lock. Granularity also affects bandwidth: writing one byte still invalidates the whole 64 B line. Sector caches split a line into sectors with separate valid bits; write-combining buffers merge multiple stores to the same line before a single BusRdX.

6.5 Memory Consistency

Coherence says each location is serialised; consistency says how *different* locations' accesses interleave as seen by different cores.

Listing 6.1: Store-buffer litmus: SC forbids (0,0), TSO allows it.

```
# Core 0                 # Core 1           # init x=y=0
SW x1,0(a0)  # x=1       SW x1,0(a1)  # y=1
LW t0,0(a1)  # y=?       LW t0,0(a0)  # x=?
# SC: at least one load sees 1. TSO: both may see 0 via store buffers.
```

Sequential Consistency (SC) requires a global interleaving respecting each core's program order where each load sees the last store. Intuitive but forbids store buffers and most load speculation.

Total Store Order (TSO, x86) allows loads to bypass earlier stores to different addresses (store buffer), but stores stay ordered with stores and loads stay ordered with loads. The litmus above can see $(0, 0)$ under TSO but not SC. x86 also has `MFENCE` to drain the buffer.

Weak / release consistency (ARM, RISC-V RVWMO) allows almost any reordering unless ordered by `FENCE` or `acquire/release`. Data-race-free programs with proper synchronisation still appear SC; racy code has no guarantee. Compilers map C++ `std::atomic` `release/acquire` to exactly these primitives.

Listing 6.2: Fences restore ordering on weak model (RISC-V).

```
# Producer (Core 0)              # Consumer (Core 1)
SW t0,0(a0)  # data=1           LW t0,0(a1)  # flag?
FENCE w,w                       FENCE r,r
SW t1,0(a1)  # flag=1           LW t1,0(a0)  # data
# Without fences, flag=1 does not imply data=1 on ARM/RISC-V.
# With fences, or using SW.rl / LW.aq, ordering is enforced.
```

Reordering	SC	TSO	ARMv8	RVWMO
Store → later Load (diff addr)	No	Yes (SB)	Yes	Yes
Store → later Store	No	No	Yes*	Yes*
Load → later Load	No	No	Yes*	Yes*
Load → later Store	No	No	Yes*	Yes*

*Allowed unless ordered by `FENCE` or `acquire/release`. TSO relaxes only Store→Load via store buffer; weak relaxes all four.

Acquire/release is lighter than a full fence: `AMOSWAP.aq` orders later ops after it, `AMOSWAP.rl` orders earlier ops before it. A message-passing pattern `data=1; flag=1` paired with `if(flag) read data` needs `FENCE W,W` after data and `FENCE R,R` before the data read, or `SW.rl flag` plus `LW.aq flag`. Place fences only at synchronisation points; they cost tens of cycles.

6.6 Coherence Performance and Speculative Interaction

Misses to shared lines stall the pipeline: a read miss to S served from LLC is about 10 ns, but a miss needing dirty M data from another core costs a cache-to-cache transfer plus acks (20–40 ns). Writes to shared lines stall in the store buffer until all sharers ack invalidations. Large fan-out sharing therefore lengthens the critical path; minimising sharing and keeping data in E/M is a first-order optimisation.

Speculative cores add a twist. A core may speculatively read a line in S, but if another core invalidates it before the speculative load commits, the load queue must detect the invalidation and replay. Store buffers sit between core and cache: a store remains buffered until its line is in M/E; only then does it become globally visible. Draining the buffer is the consistency action; invalidating sharers is the coherence action. The LSQ snoops invalidations and flushes younger speculative loads on violation.

Owned (MOESI) lets a dirty line be shared without writeback: the owner supplies data on snooped reads and remembers to write back on eviction. Forward (MESIF) picks one sharer as responder so multiple S copies do not all reply. Sector caches and write-combining buffers further reduce whole-line granularity by coalescing byte writes into one BusRdX.

Tools like `herd` and litmus suites let architects verify models exhaustively. Small programs (SB, MP, LB) are enumerated to see which outcomes hardware allows; a single unexpected $(0, 0)$ reveals a missing fence.

Key Takeaways

MSI is the minimal coherent protocol; MESI's E state eliminates needless invalidates for private data. Snooping is fast but does not scale, directory scales but adds hops, and hybrids dominate today. Coherence alone is insufficient: the consistency model (SC/TSO/weak) dictates which cross-address reorderings hardware may perform. Programmers pay fence cost only at synchronisation points to restore intuitive ordering where it matters, and they avoid false sharing by layout.

6.7 Exercises

Exercise 6.1. Trace MSI for line A initially I on both cores: C0 Rd A, C1 Rd A, C0 Wr A, C1 Rd A. List bus messages (BusRd/BusRdX/Inv), supplier, and final states. Repeat with MESI and highlight where E avoids a transaction. Which transient states appear during races?

Exercise 6.2. Snooping vs. directory: 64 cores all repeatedly read line A (shared). Compare bus traffic per miss for snooping broadcast to 63 peers vs. directory forwarding. For private-data writes (each core writes its own line), which scales better and why does MESI E matter more than directory?

Exercise 6.3. False sharing: array `int cnt[64]` holds per-thread counters (16 threads, one int each). Why does throughput collapse vs. padding each counter to 64 B? Quantify invalidations per increment and propose two fixes with trade-offs.

Exercise 6.4. Store-buffer litmus above: draw the TSO execution with store buffers where both loads see 0. Then draw the SC global order and prove $(0, 0)$ is impossible under SC but allowed under TSO. Where would a `FENCE` restore SC and at what cost?

Exercise 6.5. On RISC-V, thread 0 does `SW data=1; FENCE w,w; SW flag=1`, thread 1 does `LW flag; FENCE r,r; LW data`. Prove seeing `flag=1` implies `data=1`. Give an interleaving without fences where `flag=1, data=0` is visible and explain store-buffer reordering.

Chapter 7

Load Balance, Scheduling and Profiling

Fast cores are wasted if work is uneven, scheduled poorly, or measured wrong.

"A parallel program is only as fast as its slowest thread." Balance, placement and measurement decide whether extra cores help or hurt.

From zero: why speedup is not automatic. We start with ideal linear speedup, meet the straggler that ruins it, then build the toolkit that restores it: balancing work across cores, scheduling it with low overhead, and profiling to see where time really goes.

Learning Outcomes

After this chapter you will be able to:

(L1) compute speedup, efficiency, Karp–Flatt serial fraction and isoefficiency;

(L2) distinguish static, dynamic and guided scheduling and choose chunk size;

(L3) explain work sharing vs. work stealing with deque mechanics;

(L4) use profiling (sampling, instrumentation, tracing) to find hot spots and overheads;

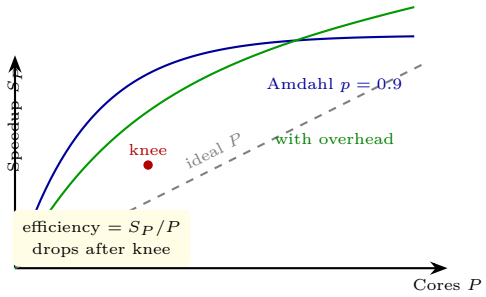
(L5) diagnose load imbalance, contention and granularity trade-offs and fix them.

7.1 Metrics: What "Faster" Means

Let T_1 be serial time and T_P time on P cores. *Speedup* $S_P = T_1/T_P$, *efficiency* $E_P = S_P/P$. Linear $S_P = P$ ($E_P = 1$) is ideal; sublinear $E_P < 1$ reveals overhead. Strong scaling fixes problem size N (Amdahl), weak scaling grows N with P (Gustafson). Real curves bend due to serial fraction, communication and imbalance.

Definition 7.1 (Karp–Flatt and isoefficiency)**.** Serial fraction $e = (1/S_P - 1/P)/(1 - 1/P)$ (Karp–Flatt). If e grows with P, overhead grows. *Isoefficiency* asks how N must grow to keep E_P constant; $N = \Theta(P \log P)$ is better than $N = \Theta(P^2)$.

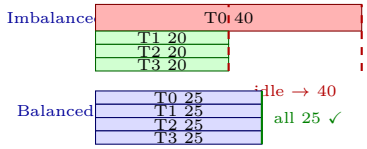
Amdahl's bound $S_P \le 1/((1-p) + p/P)$ for parallel fraction p is pessimistic when N scales. Always report T_1, T_P, S_P, E_P and variance across runs; a single median hides tail stragglers that define T_P.



The knee is where adding cores barely helps. Beyond it, communication or imbalance dominates. Isoefficiency tells you whether scaling N can push the knee right.

7.2 Load Balance

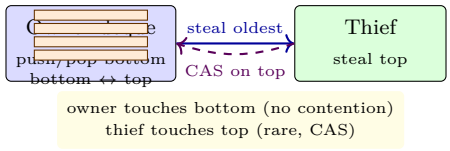
Load imbalance is when one thread gets more work, forcing others to idle at the next barrier or join. Picture a Gantt chart: four threads each should do 25 units, but the distribution is 40, 20, 20, 20. Runtime is 40, not 25; speedup collapses from 4× to 2.5× despite 100 units of work.



Causes: irregular data (sparse matrix rows with varying nonzeros), adaptive refinement, or heterogeneity (big vs. little cores). Even 10% imbalance costs 10% speedup; 50% imbalance halves it.
Static balancing partitions a priori (block, cyclic, block-cyclic). It has zero runtime overhead but fails when work per element is unpredictable. Dynamic balancing reacts at runtime: *self-scheduling* hands out chunks on demand, trading perfect balance for scheduling overhead and locality loss.

7.3 Scheduling: How Work Finds Cores

Two families: *work sharing* (scheduler pushes work to idle cores, central queue) and *work stealing* (idle cores pull from busy cores' deques). Central queues bottleneck at $P \gg 32$; stealing scales because the common path is local.



In stealing, each worker holds a double-ended queue: it pushes and pops its own tasks from the bottom (LIFO, good cache reuse), thieves steal the oldest task from the top (FIFO, large grain). Only top-bottom races need a fence or CAS. Libraries (Cilk, TBB, OpenMP tasks) use this; OpenMP `schedule(static/dynamic/guided)` gives simpler loop options.
Chunk size trades overhead vs. balance: `static` chunk N/P is minimal overhead; `dynamic,1` perfectly balances but each iteration pays queue cost and destroys locality; `guided` starts large and shrinks, a sweet spot for irregular loops. Affinity matters: stealing across sockets is dear, so hierarchical stealing prefers nearby victims first.

Listing 7.1: Loop scheduling: static vs. dynamic vs. work stealing sketch.

```python
import queue, threading

def static_schedule(N, P):
    chunk = N // P
    return [(i*chunk, min((i+1)*chunk, N)) for i in range(P)]

def dynamic_schedule(tasks, P):
    q = queue.Queue()
    for t in tasks: q.put(t)
    def worker(out):
        while True:
            try: t = q.get_nowait()
            except queue.Empty: break
            out.append(process(t))
    # dynamic: each thread pulls next chunk when free -- balances irregular cost

# Work stealing: per-worker deque, thief steals from top
# owner: push/pop bottom (fast, no CAS); thief: CAS on top (rare)
# Cilk/TBB do this; top/bottom race handled by THE protocol (Fence+CAS)
```

Granularity is the final knob: too fine (microseconds per task) and scheduling/profiling overhead dominates; too coarse (seconds) and imbalance dominates. A rule: tasks should be 10–100× scheduling cost and fit in cache.

7.4 Overheads, Granularity and Locality

Every parallelisation pays three taxes: *scheduling* (enqueue/dequeue), *communication* (coherence, message passing) and *synchronisation* (locks, barriers). If a task is 5 μs and scheduling costs 1 μs, efficiency is at most $5/6 \approx 83\%$ before any imbalance. Finer tasks improve balance but increase tax; coarser tasks cut tax but risk stragglers. The optimum sits where task time is 10–100× scheduling cost and the working set fits in private cache.

Locality matters as much as balance. A task that reuses data in L1 runs 5–20× faster than one that streams from DRAM. Guided scheduling and stealing the oldest (largest) task both preserve locality: the owner keeps hot data, the thief gets a big cold chunk worth the steal cost. Hierarchical stealing prefers victims on the same socket to avoid cross-NUMA traffic.

A practical test: vary chunk size c and measure $T_P(c)$. The curve is U-shaped: small c is overhead-dominated, large c is imbalance-dominated. The bottom of the U is your sweet spot. Tools like `perf c2c` or VTune's memory-access analysis show when false sharing or NUMA misses, not imbalance, is the bottleneck.

7.5 Profiling: Measuring What Matters

You cannot optimise what you do not measure. Three methods: *sampling* (periodic PC samples via `perf`, VTune, low overhead, statistical), *instrumentation* (insert probes at entry/exit, exact counts but perturbs), *tracing* (log every event with timestamps, detailed but huge). Hybrid tools (Score-P, TAU, NSight) combine them.

What to look for: hot spots (where time goes), stall reasons (cache miss, branch mispredict, barrier wait),

and scaling efficiency (is T_P limited by serial section, communication or imbalance?). Flame graphs aggregate samples into a visual stack; critical-path views highlight the straggler thread that defines T_P.

Listing 7.2: Karp–Flatt, isoefficiency and overhead breakdown.

```
def karp_flatt(T1, Tp, P):
    Sp = T1/Tp
    e = (1/Sp - 1/P) / (1 - 1/P)
    return Sp, Sp/P, e

T1=100
for P, Tp in [(2,55),(4,30),(8,18)]:
    Sp, Ep, e = karp_flatt(T1, Tp, P)
    print(f"P={P} Sp={Sp:.2f} Ep={Ep:.2f} e={e:.3f}")
# If e grows with P, overhead is growing -- not just Amdahl.

def overhead(T1, Tp, P): return P*Tp - T1  # serial+comm+imbalance
# Isoefficiency: find N such that overhead ~ T1 to keep Ep constant
```

Report median, p_{95} and p_{99} across ≥ 5 pinned runs; warm caches realistically. Beware observer effect: tracing a 10 μs task can double its time. Always profile the optimised baseline; a speedup over `-O0` is not a speedup.

Key Takeaways

Speedup is bounded by the slowest thread, the serial fraction and the cost of finding work. Balance removes the straggler, stealing delivers work with $O(1)$ local ops, and profiling shows which bound you hit. The loop is: measure, find the straggler or hot spot, fix balance or granularity, re-measure.

7.6 Exercises

Exercise 7.1. On $P = 8$, $T_1 = 80\,\text{s}$, $T_8 = 14\,\text{s}$. Compute S_8, E_8 and Karp–Flatt e. If $p = 0.95$ in Amdahl, what speedup does Amdahl predict? Where does the extra gap likely go (overhead vs. imbalance)? How would you test?

Exercise 7.2. Loop of $N = 10^6$ iterations with costs uniform vs. skewed (10% of iterations are 10× heavier). Compare `static`, `dynamic,1`, `dynamic,1000` and `guided` on overhead, balance and locality. Which would you pick for each cost model and why?

Exercise 7.3. Explain work stealing deque mechanics: why owner uses bottom LIFO and thief steals top FIFO, which operations need CAS/fence, and why contention is rare. Sketch a race where two thieves target the same victim.

Exercise 7.4. You trace a task-parallel program and see 40% of time in `steal` and 20% idle at barriers. Is granularity too fine or too coarse? Is balance or scheduling the culprit? Propose two fixes and how you would verify with `perf` or Score-P.

Exercise 7.5. Weak scaling: problem size doubles when P doubles, T_P stays constant. Compute scaled speedup via Gustafson for $p = 0.9$, $P = 16$. Contrast with strong scaling Amdahl for same p. When does isoefficiency $N = \Theta(P \log P)$ allow Gustafson to hold?

Chapter 8

HPC Case Studies, MapReduce and the Future

From weather forecasts to web search: how parallel patterns solve real problems, and what comes after Moore.

"A supercomputer is a device for turning a compute-bound problem into an I/O-bound problem." — Ken Batcher. Architecture, algorithm and application must co-design.

From zero: what parallel computing is for. We start with why some workloads need 10^4–10^6 cores, study three canonical HPC motifs (stencil, particle, sparse), see how MapReduce generalises the same ideas to data centres, and close with the heterogeneous, exascale and beyond-Moore horizon.

Learning Outcomes

After this chapter you will be able to:

(L1) map stencil, N-body and sparse-matrix motifs to communication patterns;

(L2) explain MapReduce (map, shuffle, reduce) and its Spark evolution;

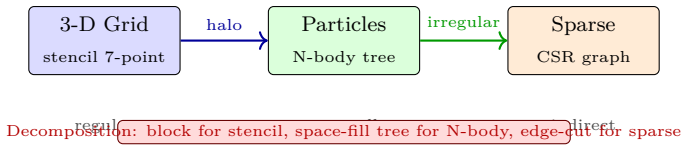
(L3) estimate isoefficiency and choose decomposition for a case study;

(L4) contrast MPI, OpenMP, CUDA and Spark on coupling and data movement;

(L5) reason about exascale power, heterogeneity and beyond-Moore directions.

8.1 HPC Motifs: Where the Cycles Go

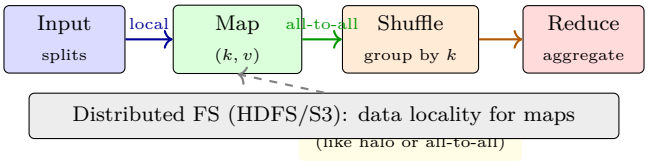
HPC codes spend 80% of time in a few motifs. **Stencil** (weather, CFD, seismic) updates each grid cell from neighbours; it is regular, nearest-neighbour communication, and halo exchange dominates. **N-body** (molecular dynamics, cosmology) computes $O(N^2)$ interactions naively, reduced to $O(N \log N)$ by Barnes–Hut or $O(N)$ by fast multipole, with irregular tree traversals. **Sparse** (graph analytics, FEM) traverses compressed structures with indirect accesses, limited by memory bandwidth and load imbalance.



Weather (ECMWF IFS) is a stencil plus spectral transforms: a 9 km global grid with 10^9 cells, halo exchange every time step, and FFTs that are all-to-all. Molecular dynamics (GROMACS, NAMD) is N-body with short-range cutoffs and long-range PME, needing both neighbour lists and 3-D FFTs. Both hit the same roofline: either memory bandwidth (sparse, indirect) or network bisection (all-to-all). Choosing block size to fit in L3 and overlapping comm with compute (non-blocking `MPI_Irecv`) is mandatory.

8.2 MapReduce and Data-Parallel Thinking

Not all parallelism is tightly coupled. **MapReduce** (Google, Hadoop) handles loosely coupled data parallelism: *map* emits (k, v) pairs per input shard, *shuffle* groups by key across the network, *reduce* aggregates per key. Fault tolerance comes from re-executing failed maps; locality comes from running maps where data lives (HDFS).



Hadoop writes intermediates to disk for fault tolerance; **Spark** keeps RDDs in memory and adds lineage for recomputation, 10–100× faster for iterative ML. Both are bulk-synchronous: stragglers dominate, so speculative execution re-runs slow tasks elsewhere, analogous to load balance (Ch. 7).

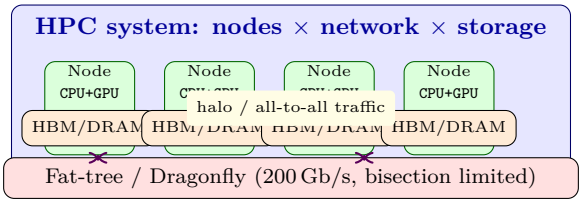
Listing 8.1: MapReduce word count in pure Python (Hadoop Streaming style).

```
from collections import Counter
import itertools

def mapper(docs):
    for doc in docs:
        for w in doc.lower().split():
            yield (w, 1)           # emit (word,1)

def shuffler(pairs):
    pairs = sorted(pairs)          # group by key (shuffle)
    for k, grp in itertools.groupby(pairs, key=lambda x: x[0]):
        yield (k, [v for _,v in grp])

def reducer(grouped):
    for k, vals in grouped:
        yield (k, sum(vals))

docs = ["Hello world hello", "world of parallel world"]
print(list(reducer(shuffler(mapper(docs)))))
# [('hello',2), ('of',1), ('parallel',1), ('world',3)]
# Spark: rdd.flatMap(...).map(lambda w:(w,1)).reduceByKey(lambda a,b:a+b)
```

When to use which: MPI/OpenMP for tightly coupled stencils needing microsecond halos; MapReduce/Spark for loosely coupled scans where seconds of shuffle are fine and fault tolerance matters. Many workloads are hybrid: ingest with Spark, train with MPI+X on GPUs.

8.3 A Cluster Sketch and Performance View

A modern HPC node: 2 sockets × 64 cores, 4–8 GPUs, 512 GB DRAM, 200 Gb/s InfiniBand. Thousands of nodes form a fat-tree or dragonfly with bisection bandwidth as the scarce resource. Storage is tiered: NVMe burst buffer, parallel FS (Lustre), and tape. Jobs are batch-scheduled (Slurm) and billed by node-hours; energy is now a first-order constraint (20–40 MW per exascale system).



Roofline still predicts: stencil is often memory-bound (AI ≈ 1–2 FLOP/B), dense N-body is compute-bound (AI > 10), sparse is memory-bound plus imbalance. Non-blocking collectives and topology-aware ranks (place neighbours together) move the workload right on the roofline.

8.4 Decomposition, Mapping and Hybrid Execution

Choosing how to split work decides communication. *Domain decomposition* (block the grid) suits stencil: each rank exchanges halos with up to 6 neighbours, $O(N^{2/3})$ surface vs. $O(N)$ volume, so weak scaling holds. *Particle decomposition* plus space-filling curves (Morton, Hilbert) suits N-body: nearby particles share a tree branch, and the curve preserves locality when assigning branches to ranks, reducing tree walk cross-talk.

Functional decomposition splits pipeline stages (e.g., ingest, simulate, analyse) and is coarse; it rarely scales alone but combines with domain splits into a 2-D process grid. Mapping ranks to the network topology matters: place neighbours on the same switch to keep halo local, scatter all-to-all participants to avoid hot spots. Tools like `hwloc` and `Slurm -distribution=block:cyclic` make this explicit.

Hybrid MPI+X reflects heterogeneity: MPI between nodes (distributed memory), OpenMP/CUDA within a node (shared memory / SIMT). A stencil might use MPI for halo, OpenMP for loops, and CUDA for the compute kernel; a Spark job might use RDDs for ETL then hand a dense matrix to MPI for a solve. The common optimisation is to overlap: post `MPI_Irecv` early, compute interior, then wait for halo; or pipeline Spark stages so shuffle of stage k overlaps map of stage $k+1$.

Energy-aware scheduling adds a new dimension. Batch systems now co-schedule power: a job that uses 80% of peak BW may be throttled via DVFS to save Watts with little time loss, improving FLOPS/W. Profiling both time and energy (via RAPL, NVML) is the profiling of the future.

8.5 Future: Heterogeneous, Exascale and Beyond

Moore's transistor scaling now advances via packaging, not density. **Chiplets** split a large die into smaller dies on an interposer (AMD EPYC, Intel Ponte Vecchio): better yield, mixed nodes (logic + HBM + I/O), but inter-chiplet BW <100 GB/s/mm and coherence across dies costs. UCIe aims to standardise die-to-die links.

Heterogeneity is the norm: CPU for branchy logic, GPU for dense throughput, NPU/TPU for matmul, FPGA for custom pipes. Programming models follow: MPI+X (X = OpenMP, CUDA, HIP, SYCL). Data movement dominates: moving a 64 B line across the machine costs $10^3\times$ a FLOP, so the mantra is move compute to data, not data to compute (near-memory, processing-in-memory via HBM stacks with logic).
Exascale (10^{18} FLOP/s) is power-limited: Frontier and Aurora are ~ 20 MW. Performance per Watt and per Joule are now primary metrics (FLOPS/W, energy-delay product). Carbon adds embodied vs. operational trade-offs. Beyond: silicon photonics for rack-scale BW, wafer-scale engines (Cerebras), and quantum accelerators for specific domains. Yet the Iron Law and Roofline remain: every optimisation must cut $IC \times CPI \times T_{clk}$ or energy×time.

Listing 8.2: Roofline + isoefficiency sketch for planning.

```
peak = 100   # GFLOP/s per node
bw   = 20    # GB/s DRAM
ridge = peak / bw  # 5 FLOP/B

def classify(ai):
    attain = min(peak, bw*ai)
    bound = "memory" if ai < ridge else "compute"
    return bound, attain

for ai, name in [(1.2,"stencil"), (12,"dense N-body"), (0.8,"sparse")]:
    print(name, classify(ai))
# Tiling stencil 1.2->6 moves from memory to compute: 24->100 GF/s win
# Isoefficiency: if overhead ~ P log P, need N ~ P log P to hold efficiency
```

The pattern across decades is constant: parallelism grows, data movement is the limiter, and the winning system minimises movement while exposing enough concurrency. Whether the device is a GPU warp or a MapReduce shard, the same questions (balance? locality? overhead?) from Ch. 7 decide success.

Key Takeaways

Stencil, N-body and sparse cover the HPC design space; MapReduce covers the data-parallel space; both are about grouping communication. Clusters are networks with compute attached, not vice versa. The future is heterogeneous and power-constrained, but the analysis tools (Roofline, Amdahl/Gustafson, isoefficiency, profiling) are unchanged.

8.6 Appendix: MPI Point-to-Point, Collectives and Halo Exchange

Distributed-memory ranks share no address space: each owns private memory and cooperates only by messages. **MPI** is the standard interface; every rank runs the same program (SPMD) and learns its identity from `MPI_Comm_rank`. Point-to-point moves data between two ranks: `MPI_Send` blocks until its buffer may be reused, `MPI_Recv` blocks until the matching message arrives. Mismatched pairings deadlock, so halo codes prefer `MPI_Sendrecv` (send and receive atomically) or non-blocking `MPI_Isend`/`MPI_Irecv` plus `MPI_Wait`, which overlaps exchange with interior computation as in Sec. 8.4. Collectives involve every rank in a communicator. `MPI_Bcast` replicates the root's buffer to all ranks (distributing parameters); `MPI_Reduce` combines values with an operator such as `MPI_SUM` or `MPI_MAX` onto the root (global residual); `MPI_Allreduce` delivers the combined result to every rank and is the workhorse of convergence tests and dot products. A tree-based broadcast costs

$O(\log P)$, not $O(P)$ sends. The **halo (ghost-cell) exchange** is the stencil motif's communication kernel from Sec. 8.1: each rank pads its block with ghost cells mirroring neighbours' edges, refreshes them every step, then computes. Surface-to-volume scaling keeps per-rank traffic at $O(N^{2/3})$. Tags disambiguate concurrent exchanges on the same communicator (tags 0/1 separate the left/right phases above), and `MPI_PROC_NULL` turns edge ranks' calls into no-ops so one code path fits every rank. Prefer `MPI_Sendrecv` or non-blocking pairs over naive Send/Recv rings, which deadlock when every rank sends first and the runtime buffers nothing. Rule of thumb: point-to-point for neighbour patterns (halos), collectives for global patterns (broadcasts, reductions); combining them, halo exchange followed by an `MPI_Allreduce` convergence check, is the standard stencil timestep.

Listing 8.3: 1-D halo exchange with MPI_Sendrecv plus an Allreduce.

```
double a[n+2]; MPI_Status st; /* a[1..n] interior, a[0]/a[n+1] halos */
int left, right;              /* neighbour ranks, MPI_PROC_NULL at edges */
for (int s = 0; s < STEPS; s++) {
  MPI_Sendrecv(&a[1], 1, MPI_DOUBLE, left, 0,   /* send left edge */
               &a[0], 1, MPI_DOUBLE, left, 0,   /* recv left halo */
               MPI_COMM_WORLD, &st);
  MPI_Sendrecv(&a[n], 1, MPI_DOUBLE, right, 1,  /* send right edge */
               &a[n+1], 1, MPI_DOUBLE, right, 1,/* recv right halo */
               MPI_COMM_WORLD, &st);
  stencil(a, n);               /* compute on interior + fresh halos */
  MPI_Allreduce(MPI_IN_PLACE, &resid, 1, MPI_DOUBLE, MPI_SUM,
                MPI_COMM_WORLD);               /* global convergence */
}
```

8.7 Exercises

Exercise 8.1. Weather stencil on a 1024^3 grid, block-decomposed $P = 64$. Estimate halo volume vs. block volume and bisection traffic per step. How does $8\times$ larger N with $8\times$ larger P (weak scaling) change the ratio? What does that imply for isoefficiency?

Exercise 8.2. Word count on 10 TB with 1000 mappers and 100 reducers. Map emits one pair per word, shuffle is all-to-all. Estimate shuffle bytes and compare to halo exchange in a stencil of same total bytes moved. Why is MapReduce's fault tolerance worth its shuffle cost for this workload but not for a tightly coupled stencil?

Exercise 8.3. Classify stencil (AI=1.5), dense N-body (AI=12) and sparse PageRank (AI=0.7) on a roofline with peak 400 GFLOP/s, BW 25 GB/s. Compute ridge AI, attainable GFLOP/s per kernel, and how tiling+cache blocking would move each point. Which kernel benefits most?

Exercise 8.4. MPI+X vs. Spark: a job does iterative k-means (10 iterations, each a map+reduce). Compare Hadoop (disk per iteration), Spark (in-memory lineage) and MPI (in-memory, no fault tolerance) on time and recovery cost if one node fails mid-job. When would you pick each?

Exercise 8.5. Exascale power: a chiplet system has 1000 nodes at 200 W each plus network 2 MW. At 1 EFLOP/s, what is FLOPS/W? If DVFS cuts voltage 20% and frequency 30%, estimate new power via $P \propto CV^2 f$ and new FLOPS/W. How do chiplets, 3-D HBM and near-memory each address the memory wall and what is one drawback of each?